Frozen Vision Encoders for Visuomotor Imitation on a Panda Reach Task

Noel Elias | visual-policy-learning | MIT License

Abstract

We study whether **off-the-shelf, frozen** vision backbones are sufficient for **image-only** behavior cloning on a tabletop Franka Panda reaching task in MuJoCo. A Jacobian IK expert provides 100 demonstrations; policies map **64x64 RGB** to **8-D** joint commands through a shared MLP head. Under identical training (Smooth L1, 200 epochs, frozen features), **CLIP ViT-B/32** achieves **100%** closed-loop success (5 episodes, seed 42), **ResNet-18 80%**, and **DINOv2-S 60%** - despite DINO using only **384-D** embeddings vs **512-D** for the others. Feature-space validation loss does not fully predict rollout success. A runtime perturbation suite (lighting, occlusion, noise) highlights encoder-dependent fragility. The imitation gap vs a **100%** IK expert (~45 steps) remains large, as expected for small offline datasets without closed-loop correction.

---

1. Problem & motivation

Visuomotor policies often assume large datasets or end-to-end fine-tuning of vision backbones. For resource-constrained labs and rapid prototyping, it matters whether **fixed pretrained representations** already encode enough geometry and semantics for simple manipulation skills when only pixels are available at deployment.

Research question: Among ResNet-18 (ImageNet), CLIP ViT-B/32, and DINOv2-S, which frozen encoder best supports behavior cloning on a randomized 3D reach target?

---

2. Method (summary)

Stage	Description
Environment	PandaReachEnv (Gymnasium + MuJoCo); success if EE-target distance < 0.05 m
Expert	Damped least-squares Jacobian IK (`ScriptedPolicy`); 100 episodes, seed 42
Observations	State-only collection, offline render from saved qpos/goals, stride-3 subsample (1,541 transitions)
Encoders	Frozen ResNet-18 (512-D), CLIP ViT-B/32 (512-D, L2 norm), DINOv2-S (384-D, L2 norm)
Learner	Shared `ActionHead` MLP on precomputed features; AdamW, cosine schedule, 80/20 sequential split

Stage	Description
Evaluation	Closed-loop rollouts, 64×64 RGB, 5 episodes per policy; robustness via Gym wrappers

Fair comparison: same action head architecture (modulo input dim), loss, schedule, and demo distribution.

---

3. Results

3.1 Open-loop (validation Smooth L1)

Encoder	Dim	Best val loss
ResNet-18	512	**0.0177**
DINOv2-S	384	0.0200
CLIP ViT-B/32	512	0.0234

Lower val loss correlates only loosely with closed-loop success.

3.2 Closed-loop (5 episodes, seed 42)

Policy	Success rate	Avg steps
CLIP	**100%**	43.0
ResNet-18	80%	56.2
DINOv2-S	60%	69.8
IK expert	100%	~45

3.3 Interpretation

- Frozen foundation features are **non-trivial** for sim reaching; encoder choice matters at deployment.
- **Smaller embeddings (DINO 384-D)** still reach useful policies - strong parameter/FLOP efficiency without catastrophic BC failure.
- **Distribution shift** (offline renders vs closed-loop physics) and **small N** limit claims; metrics need more episodes and seeds.

---

4. Limitations & proposed extensions

1. **Sample size:** ~100 demos, single eval seed - report mean +/- std over 3+ seeds and 20-50 episodes.

2. **No proprioception:** `states` in HDF5 unused; fusion may close the expert–learner gap.
3. **Frozen only:** Partial finetuning / LoRA adapters (`train_bc_e2e.py`) not benchmarked here.
4. **Robustness:** Initial robustness JSON used 1 episode per cell - re-run with `num_episodes>=20` before drawing encoder rankings under corruption.

Next experiments: DAgger or filtered BC; domain randomization during render; matched embedding dimension via projection; real-world small demo set with frozen encoder + head-only finetune.

---

5. Reproducibility

```
git clone https://github.com/noelkelias/visual-policy-learning
cd visual-policy-learning
export MUJOCO_GL=egl
pip install -e .
```

Pipeline: notebooks/data.ipynb, training.ipynb, eval.ipynb (or train_bc_frozen.py after feature HDF5). Committed artifacts: results/metrics JSON, training logs, rollout videos in videos/.

Full methodology, equations, and artifact checklist: repository README.